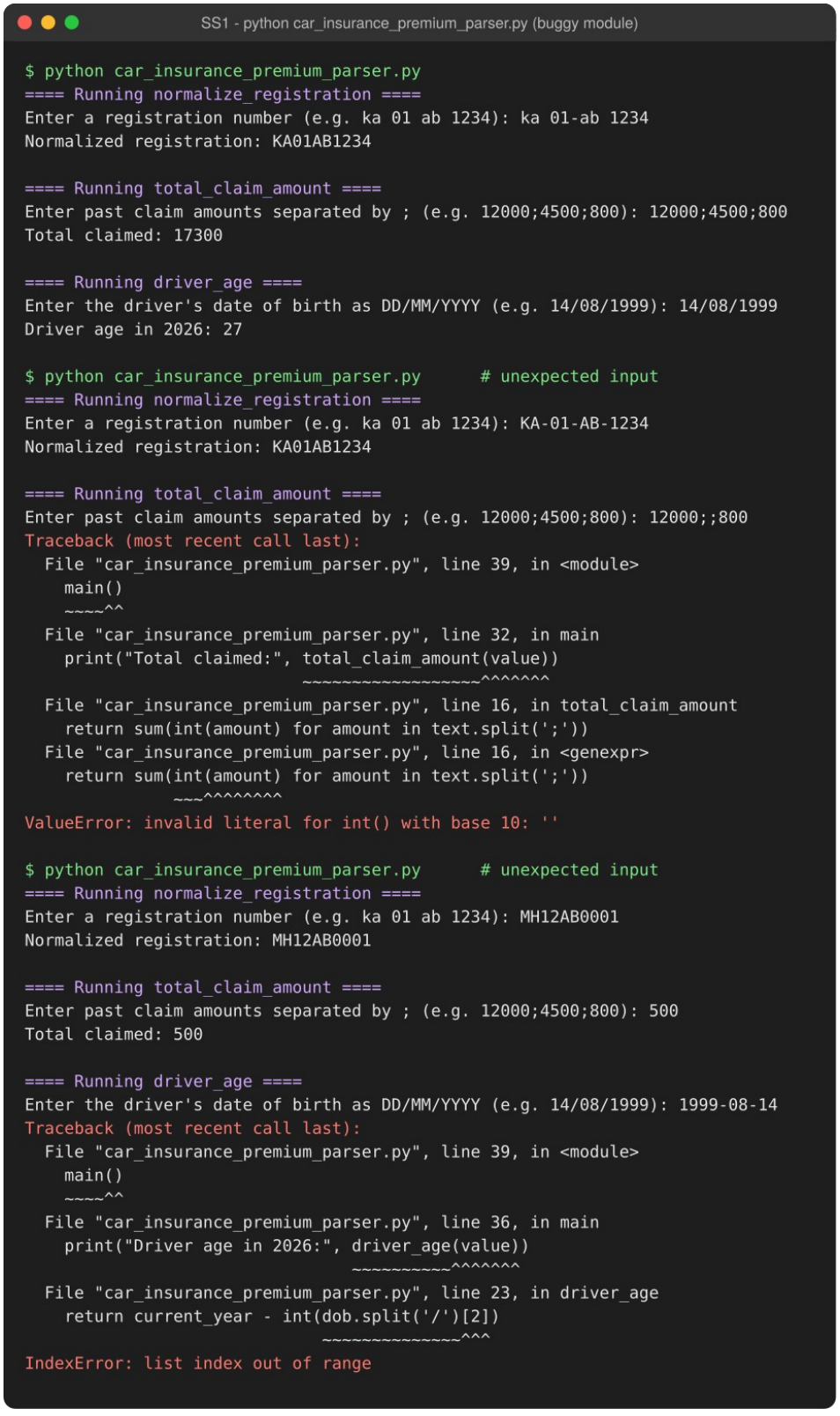
Lab 7: Fuzzing for Input Validation Bugs

Car Insurance Premium Parser (L6_L7_55)
Keshav Agrawal — PES1UG24CS719

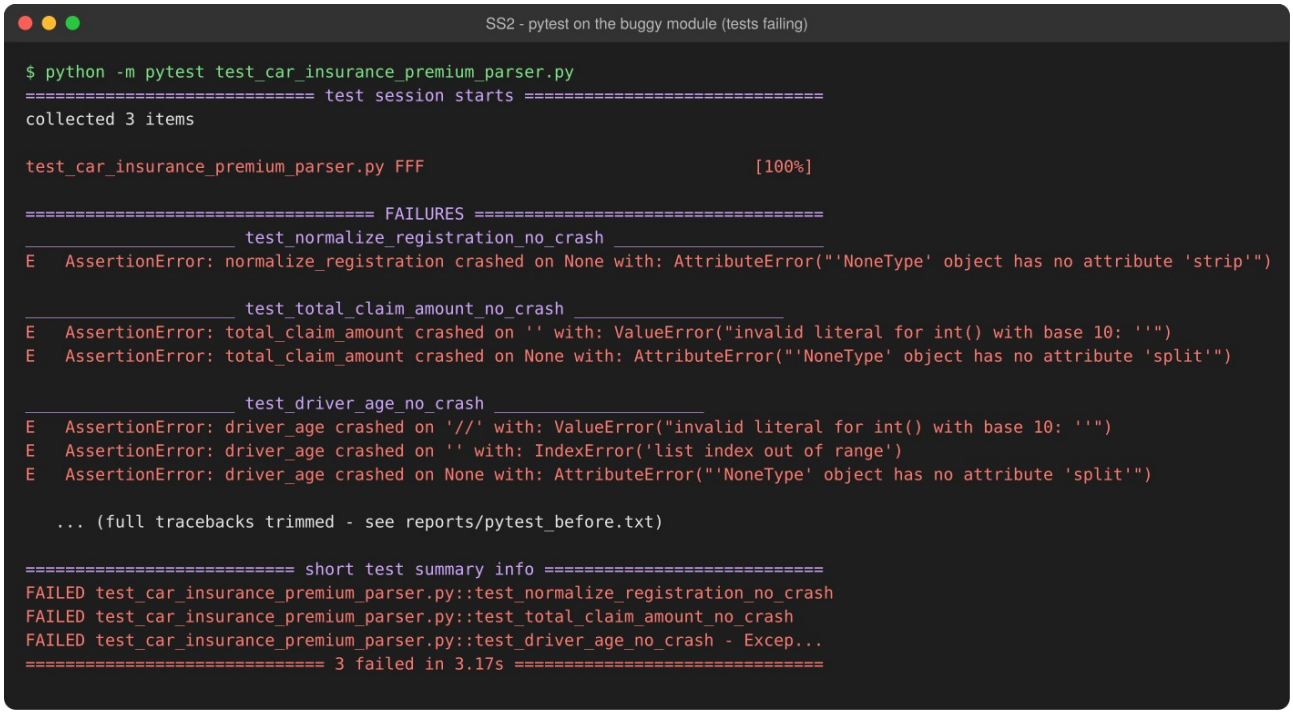
Module: `car_insurance_premium_parser.py` · Tests: `test_car_insurance_premium_parser.py` · Tools: Python 3.13, Hypothesis, pytest

SS1 — Running the buggy module



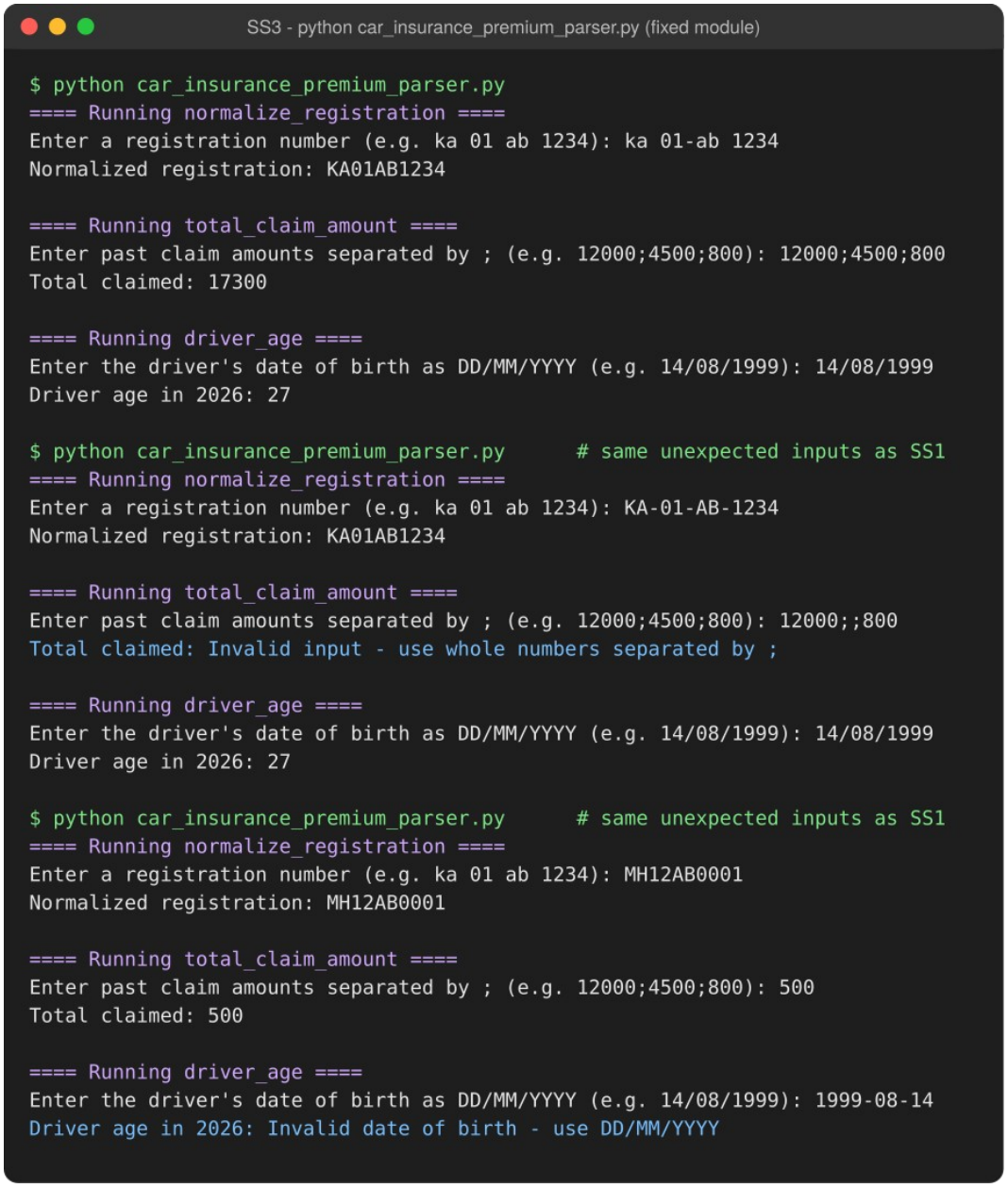
python car_insurance_premium_parser.py on the original module: works for the example input, crashes on 12000;;800 (ValueError) and 1999-08-14 (IndexError)

SS2 — Hypothesis tests failing on the buggy module



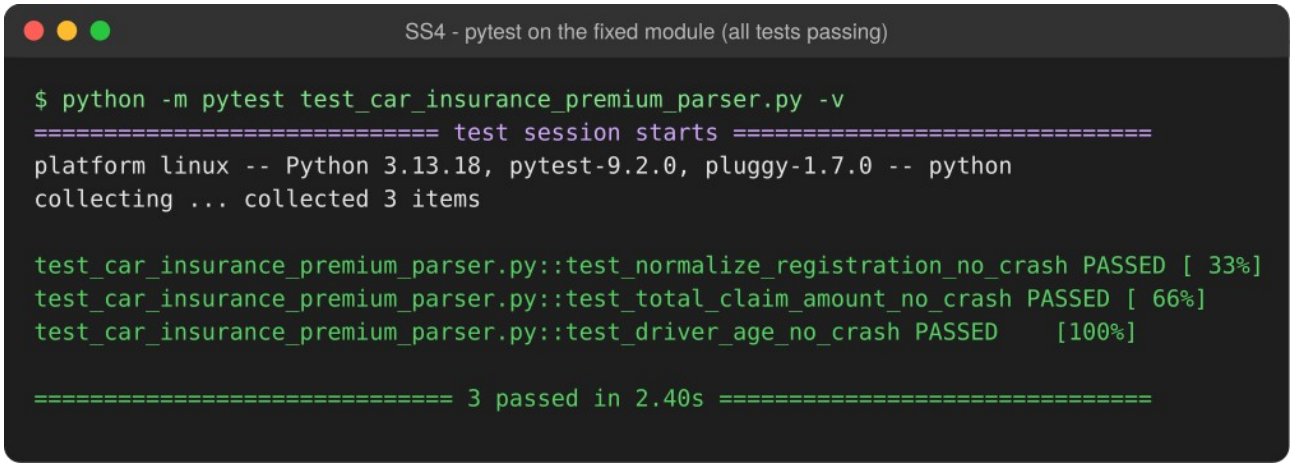
pytest test_car_insurance_premium_parser.py: all 3 tests fail; the smallest failing inputs found by Hypothesis are shown

SS3 — Running the fixed module



Fixed module: same inputs as SS1, invalid input now gives a clear message instead of a crash

SS4 — Hypothesis tests passing on the fixed module



pytest test_car_insurance_premium_parser.py -v: 3 passed

Reflection

1. What kinds of bugs did fuzzing find that normal unit tests would have missed?

All three functions crashed on input their docstrings "assumed" would never happen. Hypothesis found them automatically and shrank each one to the smallest failing case:

Function	Smallest failing input	Crash
`normalize_registration`	`None`	`AttributeError: 'NoneType' object has no attribute 'strip'`
`total_claim_amount`	`''` (also `'12000;;800'`)	`ValueError: invalid literal for int()`
`total_claim_amount`	`None`	`AttributeError: ... 'split'`
`driver_age`	`''` (also `'1999-08-14'`)	`IndexError: list index out of range`
`driver_age`	`'//'`	`ValueError: invalid literal for int()`
`driver_age`	`None`	`AttributeError: ... 'split'`

A hand-written test usually checks the example in the prompt (`12000;4500;800`, `14/08/1999`), which works. It takes a fuzzer to try an empty string, a doubled separator, a different date format or `None`. While fixing the code I also found silent logic bugs that did not crash: negative claims (`-500`) were accepted, impossible dates (`31/02/2000`) and future birth dates gave an age, and `str.isdigit()`-style checks would accept Unicode digits such as ² that `int()` then rejects.

2. How did you fix the functions (defensive programming)?

- **Check the type first:** every function returns a safe value instead of calling string methods on `None` or numbers (`''` for the registration, `None` for the claim total and the age).
- **Validate the format strictly:** claim amounts must match ASCII digits only (`[0-9]{1,9}`), which rejects empty parts, signs, decimals, Unicode digits and absurdly long numbers. The date of birth must match `DD/MM/YYYY` and then pass `datetime.strptime`, so `31/02/2000` is rejected.
- **Check business rules:** no negative claims, no future birth dates, and ages only from 0 to 120.
- **Fail clearly, not with a crash:** invalid input returns `None`, and `main()` prints a helpful message ("Invalid date of birth - use DD/MM/YYYY") instead of a traceback.

After the fix all three Hypothesis tests pass. An extra 20,000-case run with text, `None`, integers, floats, bytes and lists produced no crashes.

3. How would you use fuzzing in a real project?

I would add Hypothesis tests next to the normal unit tests for every function that parses user input (forms, CSV/JSON, query parameters) and run them in CI on every pull request, keeping the `.hypothesis` database so any failing example is re-tested until it is fixed. Fuzzing is especially valuable for security-sensitive code: a parser that crashes on unexpected input can be used for denial of service, and one that accepts malformed data can let bad values into premium calculations or the database. Property-based tests also document the contract of each function, for example "never crashes, and returns either a valid age or `None`".